Liquorice × PESSIMISTIC

SECURITY ANALYSIS

by Pessimistic

This report is public

August 6, 2026

Abstract 3
Disclaimer 3
Summary 3
General recommendations 4
Project overview 5
Project description 5
Codebase update #1 5
Codebase update #2 5
Codebase update #3 6
Audit process 7
Manual analysis 9
Critical issues 9
C01. Hard reverts in oracle logic cause DoS (fixed) 9
C02. Malicious solver can steal assets from LendingPool (fixed) 9
C03. Underwater positions cannot be liquidated (fixed) 10
Medium severity issues 11
M01. Oracle outage may block liquidations (commented) 11
M02. Liquidators can fully close undercollateralized positions for profit (fixed) 11
M03. Protocol-wide supply pause stops repayments and liquidations (fixed) 12
M04. Liquidators receive incorrect liquidation data (fixed) 12
M05. Main oracle failure does not trigger fallback logic (fixed) 12
M06. Overflow in FullMath library may block oracle (fixed) 12
M07. Interest for elapsed period uses current funding rate (fixed) 13
M08. Zero funding rate is processed incorrectly (fixed) 13
M09. Oracle-provided funding rate is used without bounds validation (fixed) 13
M10. Race condition allows solver to steal maker's assets (fixed) 13
M11. Single order execution may fail due to incorrect minFillAmount validation (fixed) 14
M12. Order signing violates EIP-712 standard (fixed) 14
M13. Stale EIP-712 domain version (fixed) 14
M14. Makers may lose unused inputs from exact-output trade (fixed) 14
M15. Phantom dust debt (commented) 15
M16. Swaps may fail abruptly instead of falling back to Uniswap V2 trade (fixed) 15
M17. Missing 0.01% V3 tier (fixed) 15
M18. Missing V3 pool check (fixed) 16
M19. Insufficient voting period validation (fixed) 16

Low severity issues 17
L01. Dead code (fixed) 17
L02. Deprecated round check (partially fixed) 17
L03. Negative TWAP tick (fixed) 17
L04. Shared default rate (fixed) 18
L05. Solvency gas scaling (commented) 18
Notes 19
N01. ETHSIGN wrapping (fixed) 19
N02. LLP/LLC DeFi compatibility (commented) 19
N03. Oracle precision loss (fixed) 20
N04. Permit2 frontrun (commented) 20
N05. Proposal boundary (fixed) 20
N06. Rate curve shape (commented) 21
N07. Shared order nonce (commented) 21
N08. Zero TWAP price (commented) 22

ABSTRACT

This report evaluates the security of the smart contracts in the Liquorice project. The objective was to identify and document security vulnerabilities within the platform's smart contracts.

DISCLAIMER

This audit makes no warranties or guarantees as to the security of the code. A single audit is insufficient to ensure complete security. We strongly recommend conducting multiple independent audits and implementing a public bug bounty program to enhance smart contract security. Additionally, this security audit should not be construed as investment advice.

SUMMARY

This report assesses the security of the Liquorice smart contract upgrade from the latest version audited by MixBytes. Large parts of the codebase were unaffected by this changes and thus left out of scope. The audit process is detailed in the following section.

The initial audit identified three critical issues: Hard reverts in oracle logic cause DoS, Malicious solver can steal assets from LendingPool, Underwater positions cannot be liquidated.

The audit also revealed many medium severity issues in multiple areas of the codebase:

- Liquidation logic (M01, M02, M03, M04)
- Oracles and price discovery (M05, M06)
- Interest rate calculations (M07, M08, M09)
- Order execution and signature validation (M10, M11, M12, M13)
- Lending operations (most of the remaining issues)

Moreover, several low severity issues and multiple notes were found.

After the initial audit, the codebase was updated. All critical, many medium severity, and some low severity issues and notes were fixed. Some issues were only partially resolved and remain present in the code and report.

In the next update, the developers fixed four medium severity issues and one low-severity issue, and commented on all remaining issues and notes.

Later, they fixed an oracle-related issue affecting sequencer-based L2 networks.

GENERAL RECOMMENDATIONS

Based on the number and nature of the identified issues, we believe the codebase would benefit from significant quality improvements.

First, the audit identified a large number of issues, including multiple medium severity findings and many notes describing edge cases, non-standard behavior, or assumptions that developers and integrators should keep in mind.

Second, several parts of the codebase appear to have evolved through multiple rounds of changes and partial refactoring. In many contracts, the implementation is more complex than the underlying logic requires. This makes the code harder to reason about, harder to modify safely, and more likely to contain subtle edge-case bugs.

As an immediate measure, we recommend documenting the resolution of all acknowledged issues, including the rationale for accepted risks, intended behavior, and any available workarounds. This documentation should be useful both internally, for future updates and maintenance, and externally, for third-party developers and integrators.

For long-term improvement, we recommend a substantial cleanup of the codebase. If the project timeline allows, the team should consider deep refactoring or a partial rewrite of the most complex components. Since the team now has a clearer understanding of the intended functionality, with the current implementation, audit findings, and test suite available as references, this work may be achievable with limited additional effort. It could materially improve maintainability, security reviewability, and long-term protocol safety.

> The development team has informed us that a refactoring of the most complex components is planned for the next protocol version.

PROJECT OVERVIEW

Project description

For this audit, we reviewed the Liquorice project from a private GitHub repository, commit 251df7c19bed65548da1ac3900d3a504602faf57.

The scope of the audit was limited to the changes introduced since commit d1f9abea12d9fe69c9a51abdb9dbfeb80ea5145e, which was previously audited by MixBytes:

```
        git diff d1f9abea12d9fe69c9a51abdb9dbfeb80ea5145e \
        251df7c19bed65548da1ac3900d3a504602faf57 --shortstat --
src/contracts/
        40 files changed, 1868 insertions(+), 896 deletions(-)
```

For context, the total number of lines of code (LOC) in Solidity contracts in the repository is 6507.

The developers shared their GitBook documentation.

The contracts will be deployed on Ethereum and Arbitrum.

At the time of the initial review, all 235 tests passed successfully, and the reported code coverage was 75.92%.

Codebase update #1

After the initial audit, the codebase was updated. For the recheck, we were provided with commit d2a43dd7015b742db61d75f3787f90b07511ab25. Our recheck was limited to the changes introduced since the previous review.

The development team added additional tests. At the time of the recheck, all 276 tests passed successfully, and the reported code coverage was 77.90%.

Codebase update #2

Later the developers provided us with a new commit 68864852c9a17b5cd9fc30f2bcabde03bc017a97 (also public version). Four medium severity issues were fixed and the rest were acknowledged and commented.

All 287 tests passed. With new tests the code coverage reached 83.47%.

Codebase update #3

The developers published a small oracle fix in the public repository at commit 270da3049ee97410d7d644ac92a218b7832aa92d.

The test suite and code coverage did not change.

AUDIT PROCESS

The audit was conducted from April 14 to May 7, 2026.

First, we reviewed the provided materials and were onboarded to the project with the help of the developers.

We used the latest audited commit as our baseline and assumed it to be issue-free for the purposes of this engagement. This approach allowed us to work within the agreed limitations and focus our review on the updated contracts. Consequently, we did not review the entire codebase; for example, the liquidation flow was outside the scope of our verification.

Throughout the audit, we maintained close communication with the development team to clarify ambiguous behavior and discuss potentially concerning aspects of the implementation.

We conducted a comprehensive manual review of all in-scope changes, assessing their logic and security properties, including:

- Core lending flows and whether user balances, debt, collateral, and interest are accounted for correctly.
- Liquidation behavior, especially edge cases with unhealthy positions, fees, callbacks, and bad debt.
- Settlement logic and solver-controlled actions, including cases where orders are partially filled or leave residual funds.
- Oracle and pricing behavior, including fallback paths, stale data, rounding, and price conversion edge cases.
- Interest-rate updates and whether new rates can affect already elapsed periods or unrelated assets.

We analyzed the project with the following automated tools:

- AI scanner AuditAgent;
- AI scanner Savant Chat.

Given the engagement limitations, we reviewed the AI scanner results selectively, focusing on findings within our audit scope.

We shared all identified issues with the development team, including findings from our manual review and AI-tool findings that we independently confirmed. We also provided the development team with the raw AI-tool reports, including potential issues outside the audit scope.

We then performed a recheck from June 23 to July 3, 2026. The development team independently reviewed the AI-tool findings and addressed a large number of the identified issues. We validated the confirmed findings, reviewed the updated code, and verified the implemented fixes. Following discussions with the development team, we downgraded the severity of several findings.

Finally, we re-ran the test suite and prepared the final report.

We performed the recheck #2 from July 14 to 20, 2026. During this period, we reviewed the applied fixes, re-ran the tests, and recalculated code coverage.

Later, the development team published the smart contracts in a public repository at commit bc609e607f300b7783b1507549dbc6d9e52aee9e. We verified that this version matches the audited code exactly.

The recheck #3 was completed on August 6, 2026. We validated the sequencer liveness check fix.

MANUAL ANALYSIS

All contracts were manually inspected to validate their logic and security. Additionally, automated analysis results were manually verified. Confirmed issues are documented below.

Critical issues

Critical issues pose severe risks to project security, potentially leading to fund loss or other catastrophic failures. Contracts should not be deployed until such issues are resolved.

C01. Hard reverts in oracle logic cause DoS (fixed)

In the **PriceProvider** contract, the `_toQuote()` function calls the quote aggregator directly and reverts when the quote price is invalid or unavailable. The primary aggregator path can also revert on failed `latestRoundData()` function calls, incomplete rounds, or failed `decimals()` function calls instead of returning `(false, 0)` and allowing the `getPrice()` function to use the fallback provider. Since solvency checks convert every non-zero debt and collateral amount through the `getPrice()` function, a temporary oracle failure can revert borrowing, withdrawals, and liquidations entirely. This can block liquidation precisely when oracle resilience is most important and allow risky positions or bad debt to remain in the protocol.

> The issue has been fixed and is not present in the latest version of the code.

C02. Malicious solver can steal assets from LendingPool (fixed)

In the **LiquoriceSettlement** contract, solver-supplied interactions are executed with the settlement contract as `msg.sender` and are not blocked from targeting registered lending pools. Since the `LendingPool.borrowFor()` function and the `withdrawFor()` function are protected only by `onlySettlement`, a malicious solver can include a **LendingPool** contract interaction during settlement and make the pool treat it as an authorized settlement action. This allows borrowing assets to an attacker-controlled receiver while assigning the debt to any maker with sufficient collateral, effectively stealing value against that maker's position.

> The issue has been fixed and is not present in the latest version of the code.

C03. Underwater positions cannot be liquidated (fixed)

In the **LendingPool** contract, successful liquidation must bring the position above the liquidation threshold. When a position accumulates enough debt, no liquidation can improve its LTV sufficiently. As a result, valid liquidations revert with `LiquidationNotSuccessful`, leaving the position impossible to unwind once it is deeply underwater. This may increase the amount of bad debt accumulated in the protocol, and there is no built-in way to get rid of it.

> The issue has been fixed and is not present in the latest version of the code.

Medium severity issues

Medium severity issues may impact the project's current functionality. This category includes bugs, potential revenue loss, operational inefficiencies, and risks related to improper system management. We strongly recommend addressing these issues.

M01. Oracle outage may block liquidations (commented)

Liquidation depends on complete pricing data for all assets in which the user has non-zero debt or locked collateral. This is important for user protection: the protocol should not liquidate a position based on incomplete or partially unavailable solvency data. However, this also means that if the price of any relevant asset is unavailable, the entire liquidation becomes impossible, even when the liquidator is only repaying and seizing other assets.

In the **LendingPool** contract, liquidation first calculates the user's LTV through `Solvency.calculateLTVs()`. This function calls `PriceProvider.getPrice()` for every relevant asset. If any of these oracle calls reverts or fails to provide a price, the LTV calculation reverts and blocks the whole liquidation. As a result, a partial oracle outage can prevent liquidations, delay risk reduction, and increase the amount of bad debt accumulated by the protocol.

> `Comment from the developers:`
> Acknowledged, won't fix (by design). Fail-closed behavior is intended: PriceProvider.getPrice() reverts only after exhausting all resilience layers (Chainlink try/catch, heartbeat/staleness checks, sequencer uptime, and a fallback oracle), so a revert means no trustworthy price exists for that asset. Since liquidation seizes collateral at oracle-derived valuations, proceeding on stale prices could wrongfully liquidate healthy positions or over-seize collateral — harm that is immediate and irreversible. The delayed-liquidation risk raised here is temporary, operationally recoverable (switch feeds or emergencyDisable), and symmetric (borrowers also can't add risk against that asset). We accept this bounded, recoverable bad-debt risk over catastrophic mispriced seizures.

M02. Liquidators can fully close undercollateralized positions for profit (fixed)

In the **LendingPool** contract, liquidation does not limit how much of a borrower's debt can be repaid and how much collateral can be seized in one liquidation. A liquidator can choose withdrawal and repay amounts that fully close a position even when it is only slightly above the liquidation threshold. This protects the protocol but creates poor UX for makers because a small temporary price move can result in a full liquidation instead of a bounded partial liquidation.

> The issue has been fixed and is not present in the latest version of the code.

M03. Protocol-wide supply pause stops repayments and liquidations (fixed)

In the **LendingPool** contract, the `_repay()` function checks the `_checkSupplyPaused()` function for the repaid asset. This means `SUPPLY_PAUSED` blocks not only new supply, but also the `repay()` function and the `repayFor()` function. The same pause also affects the `netBalances()` function, which checks both withdraw and supply pauses, and liquidation callbacks that repay debt through the `repayFor()` function. As a result, positions with debt in a supply-paused asset cannot be closed or liquidated through normal repayment paths, which can leave risky debt outstanding and may increase bad debt.

> The issue has been fixed and is not present in the latest version of the code.

M04. Liquidators receive incorrect liquidation data (fixed)

In the **LendingPool** contract, liquidation callback data can report the gross collateral withdrawn from the borrower while the liquidator receives only the net amount after protocol fees. Callback receivers such as the **LiquidationHelper** contract use the reported amount as swap input and can try to spend more tokens than they actually received. With a non-zero liquidation fee, this can revert otherwise valid liquidations.

> The issue has been fixed and is not present in the latest version of the code.

M05. Main oracle failure does not trigger fallback logic (fixed)

In the **PriceProvider** contract, the `getPrice()` function only calls the fallback provider when the `_getAggregatorPrice()` function returns `success = false`. Several primary oracle failure paths revert or return a successful zero price instead, including failed aggregator calls, incomplete rounds, quote conversion failures, and zero after normalization. These cases skip the fallback provider and can propagate into solvency checks, liquidations, and max-supply validation.

> The issue has been fixed and is not present in the latest version of the code.

M06. Overflow in FullMath library may block oracle (fixed)

In the `FullMath.mulDiv()` function, the 512-bit division path is ported to Solidity 0.8 without wrapping the Newton-Raphson inverse calculation in `unchecked`. The canonical algorithm intentionally relies on modular overflow and underflow arithmetic, but Solidity 0.8 reverts on these operations by default. If Uniswap-based quoting enters this path, price reads can revert instead of returning a quote.

> The issue has been fixed and is not present in the latest version of the code.

M07. Interest for elapsed period uses current funding rate (fixed)

In the **InterestRateModel** contract, interest for an elapsed period is charged using the newly fetched rate instead of the rate that was active during that period. This can overcharge or undercharge borrowers and creates inconsistent results between the view-only `getCurrentInterestRate()` function and the state-changing `getInterestRateAndUpdate()` function.

We recommend calculating accrued interest using the rate already stored in the contract state for the elapsed period. The newly calculated interest rate should be stored and applied only to the next accrual period. This makes interest accrual more deterministic and reduces the ability to influence accrued interest when rate updates are infrequent.

> The issue has been fixed and is not present in the latest version of the code.

M08. Zero funding rate is processed incorrectly (fixed)

In the `InterestRateModel.getInterestRateAndUpdate()` function, `r1` is updated only when the funding-rate oracle returns a non-zero value. A real market funding rate can be exactly zero, but the contract treats zero as no update and keeps the previous `r1`. If the previous value is non-zero, borrowers continue accruing interest with a stale rate until the oracle returns another non-zero value.

> The issue has been fixed and is not present in the latest version of the code.

M09. Oracle-provided funding rate is used without bounds validation (fixed)

In the **InterestRateModel** contract, the `getInterestRateAndUpdate()` function stores the funding-rate oracle value directly into `r1` without bounding its magnitude. In extreme cases, the interest-rate calculation may overflow and cause all pool operations for the affected asset to revert.

> The issue has been fixed and is not present in the latest version of the code.

M10. Race condition allows solver to steal maker's assets (fixed)

In the **LiquoriceSettlement** contract, the order's `toRepay` amount is transferred to the settlement contract before the contract calls the `LendingPool.repayFor()` function. If two orders of the same maker are filled at the same time, the first fill can reduce the maker's outstanding debt before the second fill executes its repayment. In that case, the `repayFor()` function only pulls the remaining debt amount, while the rest of the second order's `toRepay` stays in the settlement contract. This excess can later be taken through solver-controlled interactions before it is returned or supplied for the maker.

> The issue has been fixed and is not present in the latest version of the code.

M11. Single order execution may fail due to incorrect minFillAmount validation (fixed)

In the full order path, `minFillAmount` is checked against the taker's filled base-token amount. In the `settleSingle()` function, the **LiquoriceSettlement** contract instead passes `newQuoteAmount`, denominated in quote-token units, into the `_validateSingleOrder()` function and compares it with the same `minFillAmount` field. For pairs with different decimals or prices, this can reject valid partial fills and can also make full single-order fills unusable when the quote amount is numerically smaller than the base-denominated minimum.

> The issue has been fixed and is not present in the latest version of the code.

M12. Order signing violates EIP-712 standard (fixed)

In the **Signing.sol** file, the EIP-712 type hash strings omit field names for `Order`, `BaseTokenData`, `QuoteTokenData`, and `Single`. EIP-712 encodes struct types with both field types and field names, so these hashes do not match the standard typed-data representation used by wallets and off-chain tooling. This can break compatibility with signing clients, SDKs, indexers, and independent signature validators that derive hashes from the canonical EIP-712 schema.

> The issue has been fixed and is not present in the latest version of the code.

M13. Stale EIP-712 domain version (fixed)

In the **Signing.sol** file, the EIP-712 type definitions for `Order`, `BaseTokenData`, and `QuoteTokenData` were changed, but the domain version remained the same. This does not always make signatures directly reusable, because the struct hash also changes, but it breaks the expected versioning model for signed messages. Off-chain systems and signers can treat old and new message formats as the same domain version, which increases integration risk and makes signature migration less explicit.

> The issue has been fixed and is not present in the latest version of the code.

M14. Makers may lose unused inputs from exact-output trade (fixed)

In the `SwapConnector.trade()` function, exact-output swaps can consume less input than the amount transferred from the `LendingPool.rebalance()` function to the connector. The function sends only the received output token back to the user and does not refund unused input, leaving leftover tokens in the connector. Since the `trade()` function is externally callable and can operate on balances already held by the connector, stranded funds can later be swapped out by another caller.

> The issue has been fixed and is not present in the latest version of the code.

M15. Phantom dust debt (commented)

In the **Solvency** library, the `getUserBorrowAmount()` function can round dust debt shares up to 1 wei even when `totalBorrowAmount` is zero. A user left with dust shares can appear to have debt that cannot be repaid, because the pool has no borrow amount left to burn against. This can block full collateral withdrawal and leave user funds stuck.

> `Comment from the developers:`
> Acknowledged, won't fix (negligible/dust). The upward rounding in getUserBorrowAmount is intentional and is the correct direction for debt: it must never round down, or borrowers could shed real liabilities at suppliers' expense, and it follows the same ERC4626-style virtual-offset shares math used throughout the protocol. The residual is bounded to 1 wei of debt, so the only possible effect is a borrower being unable to withdraw the last few wei of collateral value backing it — an amount far below gas costs, with the rounding always in the protocol's favor and no impact on solvency. Since the consequence is limited to economically meaningless dust and never causes loss of material user funds, we accept it rather than adding special-case logic.

M16. Swaps may fail abruptly instead of falling back to Uniswap V2 trade (fixed)

In the **Swapper** contract, the `_trade()` function tries the Uniswap V3 path first and should fall back to Uniswap V2 if it fails. The catch blocks only handle `Error(string)`, so custom errors, panic errors, and empty reverts bubble up instead of triggering the V2 fallback. If the V3 path reverts with a non-string error, including local trade-integration errors, the swap fails even when the V2 path could succeed.

> The issue has been fixed and is not present in the latest version of the code.

M17. Missing 0.01% V3 tier (fixed)

In the **UniswapV3TradeIntegration** contract, pool selection checks only the 0.05%, 0.3%, and 1% Uniswap V3 fee tiers. Stablecoin pairs often concentrate liquidity in the 0.01% tier, so the router can select a much thinner pool while swaps use minimal slippage protection. Liquidations and rebalances can therefore execute through poor liquidity and lose significant value.

> The issue has been fixed and is not present in the latest version of the code.

M18. Missing V3 pool check (fixed)

In the **UniswapV3TradeIntegration** contract, routing can select a fee tier without confirming that the corresponding V3 pool exists. The V3 router can then revert with empty or non-string data, so the **Swapper** contract does not reliably reach the V2 fallback. Token pairs with V2 liquidity but no valid V3 pool can fail during liquidation or rebalance swaps.

> The issue has been fixed and is not present in the latest version of the code.

M19. Insufficient voting period validation (fixed)

In the **Configurator** contract, the constructor validates `_period` against `MAX_PERIOD` and `uint32` bounds, but does not enforce `_period >= MIN_PERIOD`. As a result, the contract can be deployed with a voting period shorter than the intended minimum, while later `setPeriod()` function calls are subject to stricter validation. This can make governance proposals execute within an unexpectedly short voting window from deployment.

> The issue has been fixed and is not present in the latest version of the code.

Low severity issues

Low severity issues do not immediately affect project operations but may introduce risks in future iterations. We recommend resolving these issues or providing justification for the chosen implementation.

L01. Dead code (fixed)

Several parts of the codebase are unused or redundant. The `Interactions._validateInteractionsAmount()` function is not called, and the lending-function selector constants in the **Interactions.sol** file are computed correctly but never used. `ILiquoriceSettlement.Order.market` is not used, validated, or included in the EIP-712 hash. The **UtilityTokensFactory** contract also accepts the lending pool address as an argument when deploying collateral and debt tokens, even though only registered lending pools can call it and `msg.sender` already identifies the pool. These items increase code surface and can confuse integrators and reviewers.

> The issue has been fixed and is not present in the latest version of the code.

L02. Deprecated round check (partially fixed)

In the `PriceProvider._getAggregatorPrice()` function, the Chainlink response is rejected with `answeredInRound < roundId` and the function reverts with `IncompleteRound`. This check is deprecated for modern Chainlink feeds and is also too brittle for the oracle fallback flow. If the primary aggregator returns such a response, the revert prevents the `getPrice()` function from trying the fallback price provider, while the safer behavior is to treat the primary price as unavailable and return `(false, 0)`.

> The check remains in the smart contracts, but it now returns `(false, 0)`.

L03. Negative TWAP tick (fixed)

In the `UniswapPriceProviderV3._consult()` function, negative average ticks are rounded using a remainder check on `uint56(tickCumulativesDelta)`. For negative deltas, this cast converts the signed value into a large unsigned value before applying the modulo operation, so the remainder check can be wrong. Negative TWAP ticks can therefore be off by one, creating a small deterministic price deviation.

> The issue has been fixed and is not present in the latest version of the code.

L04. Shared default rate (fixed)

In the **InterestRateModel** contract, assets without custom interest-rate config use the shared default config by storage reference. During an interest update, a non-zero oracle funding rate for one asset is written into the shared default `r1`. If another default-config asset later has no valid oracle rate, it can reuse that stale `r1`, causing incorrect interest accrual and potentially wrong solvency or liquidation results.

> The issue has been fixed and is not present in the latest version of the code.

L05. Solvency gas scaling (commented)

In the **LendingPool** contract, the `isSolvent()` function, the `getUserLTV()` function, and borrow validation load `AssetState` for all pool assets through the `getAssetsWithState()` function. The solvency library then iterates over all assets to find the user's debt and locked collateral balances. This scales poorly as the number of markets grows, especially when most assets are irrelevant to the user. Gas can be reduced by tracking active user assets and, with strict Maker/Lender separation, skipping solvency checks for lenders and simple-collateral data for makers.

> `Comment from the developers:`
> Acknowledged, won't fix (accepted design trade-off). Maintaining per-user active-asset lists adds its own storage writes and gas on every supply/borrow/repay, plus new bookkeeping invariants are adding extra complexity to the protocol's most safety-critical path. Since getAssetsWithState() iterates a small fixed list with no attacker-influenced growth or DoS vector, so the current cost is predictable and acceptable. We therefore prioritize the simplicity and correctness of iterating the complete asset set over a marginal, bounded gas saving.

Notes

N01. ETHSIGN wrapping (fixed)

In the **Signing.sol** file, ETHSIGN validation applies the Ethereum signed message prefix to the already computed order hash. This means off-chain signers must sign the exact pre-hashed payload expected by the contract, rather than a canonical typed-data message directly. This non-standard signing flow should be clearly documented or aligned with the expected format to avoid integration mistakes.

> The issue has been fixed and is not present in the latest version of the code.

N02. LLP/LLC DeFi compatibility (commented)

In the **CollateralToken** contract, LLP and LLC balances are represented as pool shares internally, while the `balanceOf()` function returns the current underlying-denominated amount calculated from pool accounting. At the same time, ERC20 transfers still move the raw share amount passed to the `transfer()` function or the `transferFrom()` function, and LLC transfers are restricted by the sender solvency check. This makes the tokens difficult to use safely in standard AMMs and other DeFi integrations that expect normal ERC20 balance and transfer semantics. It also limits secondary-market exits for lenders, which can be important when pool utilization reaches 100% and withdrawals are temporarily unavailable.

> `Comment from the developers:`
> Acknowledged, won't fix (no intention for AMM use). LLP and LLC are internal position-accounting receipt tokens for the lending pool — minted on supply, burned on withdraw/repay — and were never intended for AMMs or secondary markets, so their ERC20 surface exists for balance visibility and composability reads rather than third-party trading. The share-based representation (a rebasing balanceOf while transfers move raw shares) is the standard, intentional pattern for interest-accruing positions, and the solvency gate on LLC transfers is a deliberate safety invariant so collateral backing open debt cannot be moved away and left unbacked. The temporary withdrawal unavailability at 100% utilization is inherent to any lending market and is addressed by the utilization-driven interest rate model rather than secondary-market exits, so we accept this composability limitation as it carries no security impact.

N03. Oracle precision loss (fixed)

In the `PriceProvider._normalizeWithDecimals()` function, floor division is used when converting aggregator prices to quote-token decimals. For low-priced assets, especially when the aggregator has more decimals than the quote token, normalization can remove significant precision or round the price to zero. If no reliable fallback path exists, this can block price reads and should be considered before listing low-priced assets.

> The issue has been fixed and is not present in the latest version of the code.

N04. Permit2 frontrun (commented)

In the `BalanceManager.tokenPermits()` function, the `PERMIT2.permit()` function is called directly during settlement. A pending Permit2 signature can be copied from the mempool and submitted first, consuming the nonce and granting allowance to the **BalanceManager** contract. The original settlement then reverts when it tries to use the same permit, while the installed allowance can remain active until its deadline and interfere with later settlement flows.

> `Comment from the developers:`
> Acknowledged, won't fix, no fund loss, inherent permit property. The permit and the transfer are decoupled — tokenPermits() only sets an allowance, while funds move later via PERMIT2.transferFrom under the operator's control — so a replayed signature can only install the exact allowance the taker already authorized to the trusted BalanceManager, and can never be parlayed into an actual transfer. There is also no economic incentive: front-running costs the attacker gas to execute the taker's own permit for them, with no MEV, no theft, and no allowance to the attacker means pure griefing with zero payoff. Since settlement is additionally a permissioned, solver-submitted flow (typically via private/MEV-protected submission), the permit isn't exposed for opportunistic replay.

N05. Proposal boundary (fixed)

In the `Configurator.state()` function, a proposal is considered pending only when `snapshot > block.timestamp`. When `delay` is zero, `snapshot` equals the current block timestamp at creation, so the proposal becomes active in the same block. This removes the pending window and makes the `cancel()` function immediately unavailable.

> The issue has been fixed and is not present in the latest version of the code.

N06. Rate curve shape (commented)

In the `InterestRateModel._setConfig()` function, there is no check that `r1` is greater than or equal to `r0`. Since the rate on the `[0, uopt]` interval is interpolated from `r0` to `r1`, setting `r1 < r0` makes the curve decrease as utilization grows. In addition, the `getInterestRateAndUpdate()` function can replace `r1` with an oracle-controlled funding rate, which can make the effective curve shape differ from the configured risk assumptions.

> `Comment from the developers:`
>
> The finding covers two separate behaviors: one is fixed, the other is intentional design.
>
> 1. Configuration validation: fixed. _setConfig() now reverts unless r0 <= r1 <= MAX_R12, so a decreasing curve can no longer be configured. No configuration action can set r1 below r0.
> 2. Funding-rate override of r1: intentional, and doubly bounded. This interest-rate model is funding-rate-aware by design: the oracle-provided funding rate replaces r1 so the borrow rate tracks real market funding, including periods of negative or near-zero funding that a static curve cannot express. The override operates inside two independent safety bounds: (a) the oracle value is validated and capped at |r1| < MAX_R12, preventing overflow, and (b) calculateCurrentInterestRate() clamps the final rate to the governance-set [rMin, rMax] band on every call. Whatever shape the effective curve takes between r0 and the oracle-driven r1, the rate charged to borrowers can never leave the risk bounds set by governance. The configured risk assumptions are enforced at the output, and we accept the oracle override as designed.

N07. Shared order nonce (commented)

In the `Signing.validateOrder()` function, the same `Order.nonce` is invalidated for both the maker signer and the taker/effective trader. If two different makers create orders with the same nonce for the same taker, filling one order invalidates that nonce for the taker and blocks the other order. This couples maker-side and taker-side nonce spaces and can cause unexpected order cancellation.

> `Comment from the developers:`
>
> Acknowledged, won't fix (unlikely coincidence). The nonce is a full uint256 chosen per order, so for this to occur two different makers would have to independently pick the exact same value, for the same taker, with both orders live at the same time — a compound coincidence with a low probability. It cannot be forced deliberately either, since a maker cannot know or predict another maker's nonce for a given taker, so there is no way to engineer the collision as an attack. The only consequence is that one of that taker's orders would need re-quoting with a different nonce — no funds at risk and no mispricing — so we accept it rather than adding separate maker/taker nonce spaces.

N08. Zero TWAP price (commented)

In the **UniswapPriceProviderV3** contract, the `getPrice()` function returns the result of the `OracleLibrary.getQuoteAtTick()` function without checking for zero. For very low-priced assets or extreme negative ticks, the `FullMath.mulDiv()` function can floor the quote to zero. If such an asset is borrowable, its debt value can be treated as zero, enabling unsafe borrowing and bad debt.

> `Comment from the developers:`
>
> Acknowledged, won't fix (out of scope for the supported asset set).
> Flooring getQuoteAtTick() to zero requires an asset priced near the sub-wei regime (extreme low price / very negative tick), which is unreachable for the assets protocol supports. The supported set is limited to high-value, high-liquidity tokens. Also, a zero price can never reach solvency as a valid value, thus the described exploit for borrowable assets leading to unsafe borrowing and bad debt is not attainable.

This analysis was performed by Pessimistic:

Evgeny Marchenko, Senior Security Engineer
Yhtyyar Sahatov, Senior Security Engineer
Irina Vikhareva, Project Manager
Alexander Seleznev, CEO

August 6, 2026